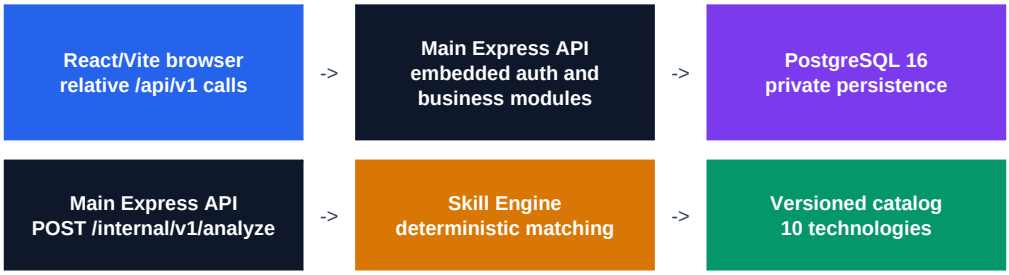
Authentication System Technical Report

Unit 3 - Activity 4 and Evidence 4: implemented topology, security, contracts, and resilience

Repository audit correction. Authentication is an embedded module of the main Express API. The independent microservice is the Skill Engine. JSearch is called by the API JobsProvider, never by the browser or Skill Engine. This edition removes routes, claims, and fallback behavior that do not exist in the source.

1. Implemented architecture and decision

DevPath uses a React/Vite browser client, a main Express API, PostgreSQL, and an independent Express Skill Engine. The API owns identity, authorization, persistence, projects, private CV storage, saved analyses, jobs provider access, and learning paths. The Skill Engine receives only analysis text and canonical skill identifiers over an internal authenticated REST call; it receives no user JWT or personal profile.



Dimension	Embedded Express auth - implemented	Standalone auth service - not implemented	External identity provider - not implemented
Boundary	Middleware and services inside apps/api	Separate deployment and service contract	Vendor-managed identity
Data control	Users, refresh families, profiles, and ownership remain in PostgreSQL	Would require cross-service identity coordination	Credentials and policy partly controlled by vendor
Operational cost	No extra service; API verifies RS256 locally	Additional health, network, and deployment surface	Vendor dependency and pricing constraints
Verified choice	Matches repository topology and current scope	Not present in DevPath	Not present in DevPath

Architectural conclusion

The embedded module is appropriate for the current monolith-plus-engine topology. It must not be presented as a third microservice. The Skill Engine is the independently deployed internal microservice used by the application.

2. Authentication workflows

2.1 Registration and login

Registration validates name, normalized email, and password; hashes the password with Argon2id; inserts the account, profile, session family, and hashed refresh token transactionally; then returns a short-lived access token and sets an opaque refresh cookie. Login performs an Argon2id verification even on the missing-account path to reduce account-enumeration timing differences.

Step	Caller	Implemented operation	Result
1	Browser	POST /api/v1/auth/register or /auth/login	Validated JSON over the public API
2	Express auth module	Argon2id hash/verify; parameterized PostgreSQL queries	Account or 401/409 structured error
3	Express auth module	Sign RS256 access JWT; create random refresh token	200/201 JSON plus HttpOnly cookie

2.2 Protected request and analysis

Hop	Authentication and data boundary
Browser -> API	Authorization: Bearer <access JWT>. Middleware verifies RS256, typ, issuer, audience, expiry, candidate role, UUID subject, and UUID jti.
API -> PostgreSQL	Owner identity comes only from the verified subject. SQL filters by user_id; foreign resources return 404.
API -> Skill Engine	POST /internal/v1/analyze with a separate service Bearer secret and X-Request-Id. Body contains source text and skill IDs, not user identity.
Skill Engine -> API	Bounded JSON response with required/matched/missing IDs, percentage, warnings, and evidence. API validates invariants before persistence.

2.3 Refresh rotation and logout

POST /api/v1/auth/refresh consumes the HttpOnly cookie, locks the stored SHA-256 token hash, rejects expiry or revocation, detects reuse, rotates once, and preserves the original seven-day absolute expiry. Reuse revokes the refresh family. POST /api/v1/auth/logout revokes the family when a token is present and clears the cookie. An issued access token remains valid only until its 15-minute expiry.

Control	Implemented value
Password hashing	Argon2id; memoryCost 19456, timeCost 2, parallelism 1, hashLength 32
Access token	RS256; RSA key at least 2048 bits; 900 seconds; stored only in browser memory
Refresh token	32 random bytes encoded base64url; only SHA-256 hash stored; seven-day absolute expiry
Cookie	HttpOnly, SameSite=Strict, Path=/api/v1/auth; Secure outside explicit local HTTP

3. JWT claims and authorization model

The access token does not contain email, password, profile data, recruiter/admin roles, or provider credentials. DevPath currently accepts the single application role candidate and authorizes records by the verified subject.

```
{ "alg": "RS256", "typ": "at+jwt", "sub": "<user UUID>", "role": "candidate", "iss": "urn:devpath:auth", "aud":
"devpath-api", "iat": <unix seconds>, "exp": <iat + 900>, "jti": "<UUID>" }
```

4. Public and internal interface contracts

Boundary	Route	Auth	Representative outcomes
Browser -> API	POST /api/v1/auth/register	Public	201; 400; 409; 413; 429
Browser -> API	POST /api/v1/auth/login	Public	200; 400; 401; 429
Browser -> API	POST /api/v1/auth/refresh	Refresh cookie	200; 401; 403; 429; 503
Browser -> API	POST /api/v1/analyses	Access JWT	200 replay; 201; 400; 401; 409; 503; 504
Browser -> API	GET /api/v1/jobs and /jobs/details	Access JWT	200; 400; 401; 404; 429; 503; 504
API -> engine	POST /internal/v1/analyze	Service Bearer	200; 400; 401; 413; 422; 500
API -> JSearch	GET /search-v2 and /job-details	X-RapidAPI-Key	Normalized success; 404; 429; timeout/provider failure

The API also protects profile and skills, projects CRUD, private CV upload/download/delete, analysis history, and learning path routes. OpenAPI documents and 54 validated JSON examples define successful and error shapes. The M3/M4 filenames retain their historical proposed label, but the corresponding accepted routes are implemented and regression-tested in v0.4.0.

JSearch safety boundary

JobsProvider is part of the main API. fixture mode is the safe default and is visibly labeled. live mode requires an explicit enable flag and a backend-only RAPIDAPI_KEY. The host and paths are fixed, requests are bounded by timeout and maximum response size, provider cursors are encrypted before exposure, and 429 is not automatically retried. A live provider failure returns a structured error; the implementation does not silently switch to fixture data.

5. Implemented component topology

API-owned module	Responsibility	External/internal dependency
Authentication	Registration, login, RS256 verification, refresh rotation, logout	PostgreSQL only
Profiles and analyses	Owner skills, snapshots, history, evidence, idempotency	PostgreSQL and Skill Engine
Projects and private CV	Owner CRUD, PDF validation, private storage metadata	PostgreSQL and private volume
JobsProvider	Fixture/live search, detail, normalization, encrypted cursor, rate/timeout mapping	JSearch from API only
Learning paths	Deterministic generation, steps, progress, deleted-project snapshot	PostgreSQL and curated catalog

6. Failure behavior and recovery

Failure	Verified behavior	Recovery / data guarantee
Invalid/expired access JWT	401 UNAUTHENTICATED; no protected handler runs	Browser may attempt one refresh and retry
PostgreSQL unavailable	Readiness and stateful operations fail; internal details are not returned	503-style structured response; no fabricated success
Skill Engine unavailable/slow	Analysis returns 503 or 504; draft and idempotency key remain	No analysis row persisted; safe retry creates once
JSearch 429/timeout/invalid JSON	Mapped to bounded 429/504/503 responses without key leakage	No hidden retry and no live-to-fixture substitution
Project deleted	Linked learning step keeps the stored activity snapshot and removes broken relation	Learning path remains readable

Error responses contain code, safe message, requestId, and optional field errors. Unexpected server errors are logged with request context, while the client receives a generic INTERNAL_ERROR without stack traces, SQL, filesystem paths, tokens, or provider credentials.

Compose and runtime isolation

Docker Compose publishes only the web/proxy entry point. PostgreSQL, API, and Skill Engine remain on private networks as configured; the engine internal route is not exposed through the public proxy. The API container runs as a non-root identity aligned with the read-only private-key owner. PostgreSQL 16 runtime probes cover startup, proxy, registration, analysis, persistence after restart, M2/M4 persistence, and engine recovery.

7. Applied security controls

Risk	Implemented control	Repository evidence
Credential compromise	Argon2id; dummy verification path; refresh hashes only	auth-service.ts and auth repository integration tests
Token forgery/replay	RS256 with issuer/audience/typ checks; 15-minute access; single-use refresh families	auth-service.ts and auth tests
Broken access control	Owner from verified JWT subject; owner-filtered SQL; foreign resources return 404	projects, CV, analyses, and learning-path repositories/tests
Injection and XSS	Zod validation, parameterized pg queries, HTML rendered as text, safe external links	API/web unit tests and Chromium E2E
Secret exposure	Ignored local env/keys; backend-only provider key; sanitized error bodies and CI artifacts	.gitignore, setup-local.mjs, provider and error-handler tests
External dependency abuse	Fixed JSearch origin, timeout/size bounds, explicit live gate, no 429 retry	provider.ts and controlled HTTP integration tests

8. Evidence and reproducibility

Artifact	Verified purpose
packages/contracts/openapi/*.yaml	Public, M2, jobs, learning, and engine interfaces with examples and error responses
apps/api/requests/e3-jsearch.http	Reproducible E3 public requests without embedding credentials
docs/acceptance/M3-results.md	Separates fixture tests from the bounded real JSearch verification
tests/e2e/*.spec.ts	Chromium journeys with real API, Skill Engine, and isolated PostgreSQL
docs/acceptance/probes/ops-compose.mjs	Compose startup, persistence, recovery, and proxy runtime checks
GitHub Actions	Lint, format, typecheck, build, tests, PostgreSQL 16 E2E, docs, and Compose

Verified conclusion

DevPath implements authentication inside the main Express API and uses the Skill Engine as its independent microservice. The system has more than three REST operations, consistent JSON errors, isolated owner data, versioned contracts, and automated success/error coverage. JSearch live access was verified separately and remains disabled by default to prevent accidental quota use. This report intentionally does not claim a separate authentication microservice, browser access to secrets, silent provider fallback, or routes absent from the repository.

Audit basis: DevPath v0.4.0 release commit cacec37bba40528cfc091888bd2a33290b414940 and synchronized develop commit bc104f044a65f5102cbf0c8fb46c8c37649e58f9. Generated from repository source by scripts/generate-auth-report.py.